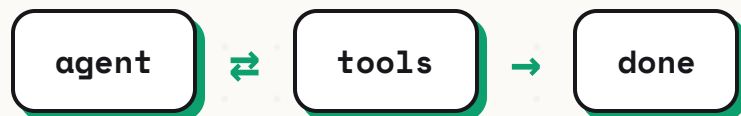


LangGraph

Build **stateful, multi-step agents** as a graph — nodes that do work, edges that decide what's next, and shared state flowing through. Loops included.



Handwritten notes: A large bracket on the right side of the slide, and the phrase "multi step" written diagonally with an arrow pointing towards the flow diagram.

Agents don't run in a **straight line**

A real agent needs to...

- **Loop** — call a tool, see the result, call another.
- **Branch** — take a different path based on what it found.
- Carry **state** across many steps.
- Pause for a **human**, then resume.

A chain can't do this cleanly

- DAGs go one direction — no cycles.
- Persistence & memory are bolted on.
- Control flow logic leaks everywhere.

LangGraph's answer: model the agent as a *graph* — an explicit state machine you can loop, branch, pause, resume, and inspect.

— DEFINITION

What is LangGraph?

A library from the LangChain team to build **stateful, multi-step** LLM workflows as a graph. Three primitives:

STATE

Shared state

A typed dict every node reads and updates.

NODE

Nodes

Functions that do a step of work.

EDGE

Edges

Control flow — including **conditional** ones.

Key move: control flow becomes *data* — a graph you can draw, replay, checkpoint and interrupt. It runs LangChain components inside the nodes.

Seven ideas that **unlock agents**

State

Typed, shared, merged by reducers (e.g. append messages).

Nodes

Do work, return a partial state update.

Edges

START → ... → END; wire the order.

Conditional edges

Route based on state — the branch.

Cycles

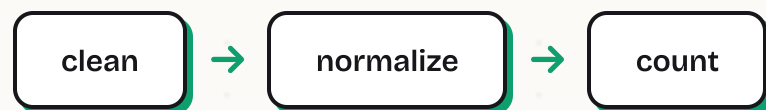
Edges that loop back — the **agent loop**.

Checkpoint

Persist state → memory, resume, threads.

+ **Interrupts**: pause at a node, hand off to a human, resume with their decision — human-in-the-loop, built in.

Nodes update **shared state**



Each node returns a partial update; LangGraph merges it into the one shared **State**. Edges define the order.

```
01_state_graph_basic.py

class State(TypedDict):
    text: str; words: int; steps: list

def clean(s): return {"text": s["text"].strip()}

g = StateGraph(State)
g.add_node("clean", clean)
g.add_edge(START, "clean")
g.add_edge("clean", END)
app = g.compile()
app.invoke({"text": " Hi ", ...})
```

Runs offline – no LLM – just to show the mechanic.

The agent loop



`tools_condition` is the conditional edge: if the model asked for tools
 → go to **tools**; else → **END**. After tools, loop back to **agent**. That cycle is
 a real ReAct agent.

```

02_agent_loop.py

llm = ChatOpenAI(...).bind_tools(tools)
def agent(s): return {"messages": [llm.invoke(s["messages"])]}

g.add_node("agent", agent)
g.add_node("tools", ToolNode(tools))
g.add_edge(START, "agent")
g.add_conditional_edges("agent", tools_condition)
g.add_edge("tools", "agent") # the loop
  
```

`get_pods` → `get_pod_logs` → root cause — it loops until done.

What the graph **buys** you



Cycles

Retry, reflect, self-correct — loop until a condition is met.



Branching

Route to different nodes based on state.



Persistence

Checkpointers save state → memory, threads, resume.



Human-in-loop

Interrupt, wait for approval, resume.



Streaming

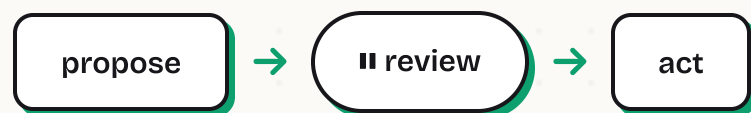
Stream node-by-node so you can see it think.



Multi-agent

Nodes can be whole agents — orchestrate a team.

Pause for a human



`interrupt()` pauses the graph and persists it. A human decides; you `resume` the same thread with their answer. This is the **propose** → **approve** → **act** pattern — the exact FinOps approval flow.

```
03_human_in_the_loop.py

def review(s):
    d = interrupt({"action": s["resource"],
                  "question": "Approve?"}) # pause
    return {"approved": d == "approve"}

app = g.compile(checkpointer=MemorySaver())
app.invoke(inputs, cfg) # runs → pauses
app.invoke(Command(resume="approve"), cfg) # resume
```

Runs offline — the mechanic without a model.

Where graphs **win**

AGENT

Multi-tool ReAct agents

Investigate, act, retry — DevOps, research, coding assistants.

APPROVE

Human approval workflows

Propose → notify → approve → act (our FinOps portal).

TEAM

Multi-agent orchestration

Planner + workers + critic under one graph.

STATE

Long-running assistants

Persistent memory across sessions via threads.

Sweet spot: anything that *loops, branches, remembers, or waits* — the moment "one pass" isn't enough.

— COMPARISON

LangGraph vs LangChain

	LANGCHAIN	LANGGRAPH
Model	pipeline / DAG	graph / state machine
Flow	linear, one direction	loops, branches, conditionals
State	implicit, passed along	explicit, typed, shared
Memory	add-on	built-in checkpointer
Human-in-loop	manual	native interrupt()
Best for	prompts · RAG · extraction	agents · retries · approvals

Not either/or: LangGraph orchestrates; LangChain provides the parts. A LangGraph node usually calls a LangChain model, tool, or retriever.

When to graph, and **run it**

Choose LangGraph when

- You need a loop (agent that retries/reflects).
- Branching or shared, typed state.
- Memory across turns, or a human approval.

terminal

```
python langgraph/01_state_graph_basic.py # graph model
python langgraph/02_agent_loop.py       # the loop
python langgraph/03_human_in_the_loop.py # pause/resume
```

01 & 03 run offline; 02 needs OPENAI_API_KEY.

LangGraph, in a breath

- Build agents as a **graph**: State + Nodes + Edges.
- Conditional edges + cycles give you the **agent loop**.
- **Checkpoint** = memory & resume; **interrupt()** = human-in-the-loop.
- Use it for **agents, retries, approvals, multi-agent** — where a chain can't.

One line to remember: LangChain composes a *line*; LangGraph orchestrates a *loop* — with state and a human when you need one.